

PRD: AI-Powered Assessment Generator for Course Admins

1. Overview

Course admins today build every assessment by hand — writing questions, structuring answer options, and placing them on the course outline one at a time. This is slow, repetitive, and scales poorly as course libraries grow. This ticket introduces an AI-powered assessment generator that reads existing lesson transcripts and produces draft assessments (across multiple question types) directly on the course outline, dramatically compressing the time from "course content exists" to "course is fully assessable."

This is a **prototype-stage** feature. The goal is to validate the core loop — transcript in, assessments out, admin reviews — before investing in editing, fine-tuning, or advanced prompt controls.

2. Jobs To Be Done

Main Job

When I have a course with lesson content already created, I want to quickly produce relevant, varied assessments so I can make the course ready for learners without spending hours writing questions manually.

Job Map

Stage	What the Admin Does Today	Where It Breaks Down
Define	Decides which lessons need assessments and what types	No guidance on which question types suit which content; decisions are ad hoc
Locate	Opens each lesson, re-reads content to extract key concepts	Mentally context-switches between "reading" and "question writing"
Prepare	Drafts questions, writes answer options, creates distractors	Most time-consuming step; distractors are especially hard to write well
Confirm	Reviews own work (self-review only)	No second pair of eyes; no way to know if questions align to learning objectives
Execute	Manually places each assessment on the course outline	Tedious drag-and-drop or form-fill for every single question
Monitor	Checks learner performance post-launch	No feedback loop into question quality

Resolve	Edits or removes poor questions after learner complaints	Reactive, not proactive
----------------	--	-------------------------

The AI generator collapses **Locate** → **Prepare** → **Execute** into a single action and upgrades **Confirm** by giving admins a clear review moment before assessments go live.

Related Jobs

- **Ensure assessment diversity:** Admins want a mix of question types (recall, application, recognition) so learners are tested at multiple cognitive levels.
- **Keep course outline coherent:** Generated assessments must land in sensible positions on the outline, not create structural chaos.
- **Maintain trust in content quality:** Admins need confidence that AI-generated questions are grounded in actual lesson material, not hallucinated.

Emotional & Social Dimensions

- **Emotional:** Admins want to feel *relieved* ("this used to take me a full day") and *confident* ("these questions are solid enough to publish"). They do **not** want to feel like they're babysitting an unreliable tool.
- **Social:** Admins want to be seen as professionals who produce polished courses. If learners encounter nonsensical or irrelevant questions, it reflects poorly on the admin. The AI must not undermine that professional image.

3. Goals

1. **Reduce assessment creation time by an order of magnitude** — from hours of manual writing to minutes of review and curation.
2. **Ground all generated assessments in actual lesson content** — zero tolerance for questions about material not covered in the transcripts.
3. **Support all eight assessment types at launch** — Single choice, Match the pairs, Sequence, Categorise, Fill in the blanks, Short text, Exercise, Poll — plus the distinct Situational tests type.
4. **Preserve admin agency** — the admin controls *what types* are generated and can delete or regenerate; the AI is a drafter, not a decision-maker.
5. **Place assessments directly on the course outline** — no copy-paste, no export/import, no disconnected quiz builder.

4. Job Stories

Selection & Generation

- **When** I open a course that has lesson transcripts available, **I want to** trigger assessment generation with a single action, **so I can** get draft assessments without re-reading every lesson myself.
- **When** I'm about to generate assessments, **I want to** pick which assessment types the AI should use, **so I can** ensure the output matches my pedagogical intent (e.g., heavy on scenario-based questions for a compliance course).
- **When** my course has only some lessons with transcripts, **I want to** see clearly which lessons will be covered and which will be skipped, **so I can** decide whether to proceed now or add more transcripts first.

Review & Curation

- **When** the AI has finished generating, **I want to** see all proposed assessments in context on the course outline, **so I can** evaluate them where they'll actually live rather than in a disconnected list.
- **When** I spot a generated assessment that is poor quality or irrelevant, **I want to** delete it immediately, **so I can** keep my course clean without editing overhead.
- **When** I'm unsatisfied with the overall generation output, **I want to** regenerate a fresh set, **so I can** get a second draft without manually undoing anything.

Edge Cases & Trust

- **When** none of my lessons have transcripts, **I want to** understand why generation isn't available and what I need to do, **so I can** unblock myself without contacting support.
- **When** I see a generated assessment, **I want to** know it came from AI (not a human author), **so I can** apply the right level of scrutiny before it reaches learners.

5. Requirements

Functional Requirements

ID	Requirement	Priority	Notes
FR-01	The system must support generating nine assessment types: Single choice, Match the pairs, Sequence, Categorise, Fill in the blanks, Short text, Exercise, Poll, and Situational tests.	Must	Situational tests have a distinct structure (scenario + decision branches).
FR-02	The admin selects desired assessment types before generation; the AI determines the quantity	Must	No quantity picker — the AI decides based on transcript

	of each.		richness.
FR-03	Generation is sourced exclusively from lesson transcripts within the course.	Must	RAG-style grounding; no external knowledge or arbitrary prompts.
FR-04	When a course has partial transcript coverage, the system generates from available transcripts and surfaces a clear message (e.g., "3 of 5 lessons have transcripts — assessments will be generated from those 3").	Must	Do not block generation on partial coverage.
FR-05	When zero lessons have transcripts, the system shows an empty state explaining why generation is unavailable and what the admin should do.	Must	Must be self-service — admin can unblock by adding transcripts.
FR-06	Generated assessments are placed directly on the course outline.	Must	No intermediate staging area or clipboard.
FR-07	Each generated assessment is visually labeled as AI-generated.	Must	Transparency labeling per research best practices.
FR-08	The admin can delete any individual generated assessment from the outline. Deletion is immediate, with a short undo window on the confirmation toast — no confirmation dialog.	Must	Friction belongs where loss is real. A modal per question makes curating a 20-item batch punishing (Linear's undo warnings).
FR-09a	The admin can regenerate a single assessment in place — "Delete & regenerate" — replacing just that item with a fresh AI draft.	Must	Keeps deletion from being punishing without introducing editing (Delete and redraft). One weak question no longer costs the whole batch.
FR-09b	The admin can regenerate the whole set, replacing all AI-generated assessments. This requires confirmation, and the confirmation makes the blast radius visible — the affected assessments are highlighted while the rest of the outline recedes.	Must	High-loss action, so it earns a dialog (T-Mobile delete focus). Manually created assessments are never touched.
FR-10	No in-place editing of generated assessments is supported.	Must	Scope constraint for prototype. Admin's options are keep, delete, or regenerate.
FR-11	A progress indicator is shown during generation, naming the lesson currently being read and counting elapsed time upward.	Should	Generation time varies with transcript length, so no honest ETA exists. Counting up proves the process isn't stuck (Icons8).
FR-12	The system logs which lessons were used as source material for each generated assessment.	Should	Enables future traceability and trust features.

6. Research & Best Practices

Synthesis

Grounding AI output in course content is non-negotiable. Research on AI-generated MCQs in health science education found that approximately 57% of AI-generated questions are directly usable and 31% require minor edits, but key failure modes include hallucinations, poor distractors, and misalignment with learning objectives. The study recommends grounding questions in specific course material (RAG-style), mandatory human review, and starting with low-stakes formative assessments before scaling ([PMC — AI-Generated MCQs in Health Science Education](#)). This directly validates the ticket's approach of generating from lesson transcripts rather than arbitrary prompts.

The "approve structure first" principle applies here. Coursiv's analysis of AI course creation emphasises that "fixing the outline takes minutes; fixing forty lessons built on a bad outline takes days." They recommend separating AI lesson generation from quiz generation so admins can accept one and redo the other independently, and treating AI facts as claims to check ([Coursiv — AI for Course Creation](#)). In our case, this maps to: let admins approve the course outline before generating assessments, and make regeneration cheap.

The AI paradigm is intent-based, not command-based. NN/g's research on the AI paradigm argues that AI represents an intent-based interaction model where users specify *what* they want, not *how*. However, critical gaps exist: iterative refinement is poorly supported in most tools, and users struggle to identify errors when they can't see how something was produced. They recommend hybrid UIs combining intent-based generation with GUI controls ([NN/g — The AI Paradigm](#)). Our design addresses this by giving admins a GUI control (assessment type picker) before triggering intent-based generation.

AI UX must include error recovery and transparency. Koru UX identifies six AI UX patterns including Refine Output via contextual menus (regenerate, adjust), Human Verified vs. AI-Generated badges, Explainability Layers for trust, and Error Recovery with undo/redo. A critical anti-pattern is treating AI output as final with no revision path ([Koru UX — AI Patterns for UI Design](#)). Our delete-or-regenerate model provides the minimum viable error recovery path.

Drafting and status indicators reduce AI mystery. UX Studio identifies six patterns for AI in SaaS: Drafting & Versioning (present AI drafts, not blank slates), Status Indicators (layered workflow states such as processing → draft ready → pending review), Review Interfaces (show AI output alongside source material), and Source Citations (link generated content back to originals) ([UX Studio — AI Design Patterns in SaaS](#)). The progress indicator (FR-11) and AI-generated label (FR-07) directly implement these patterns.

State-of-the-art LMS platforms already offer AI assessment generation. D2L's Lumi (powered by Anthropic's Claude) provides 14+ AI capabilities including assessment generation aligned to Bloom's Taxonomy levels and specific learning outcomes, with questions generated from course content rather than arbitrary prompts ([D2L — AI LMS Platforms Compared](#)). This establishes the competitive baseline.

Assessment type selection should be intentional. eLearning Industry's guide to interactive quiz formats covers matching pairs, sequencing, categorisation, and fill-in-the-blank, recommending keeping matching pairs at 5–10 items, limiting drag-and-drop to 8–12 options, applying fill-in-the-blanks only for

unambiguous answers, and using scenario-based formats for real-world decision-making ([eLearning Industry — 8 Types of Interactive Quiz Formats](#)). LearnExperts further emphasises choosing question types based on cognitive level: recognition (MCQ), recall (fill-in-blank), application (scenario/situational) ([LearnExperts — Assessment Questions: How to Choose](#)). These constraints should inform the AI's generation logic.

Placing assessments directly on the outline avoids a major pitfall. MindStudio notes that a major problem with standalone AI quiz generators is that they create disconnected assessments requiring copy-paste into the LMS. The ticket's approach of placing assessments directly on the course outline avoids this friction entirely ([MindStudio — Build AI-Powered Quizzes](#)).

Bulk actions need careful UX. Eleken's guidelines for bulk action UX recommend visible checkboxes (24×24px minimum), showing action buttons only after selection, explicit confirmation for destructive actions, per-item failure reasons, and undo via toast notifications ([Eleken — Bulk Action UX](#)). This applies to the bulk delete/regenerate flows.

UX References

App	Flow / Screen	Mobbin URL	Pattern Description	Relevance
Kajabi	Adding & Creating	View	Quiz/assessment creation builder with type selection	Direct competitor reference for how a course platform structures assessment creation
Teachable	Adding & Creating	View	Assessment builder with button/selection UI	Competitor LMS showing question type picker and creation flow
Podia	Adding & Creating	View	Quiz builder interface	Simplified quiz creation pattern for course creators
Typeform	Adding & Creating	View	Form/quiz builder with question type selection	Industry-standard pattern for multi-type question creation UI
Profound	Editing & Updating	View	AI content generation/review workflow	AI-powered content editing with review interface — reference for the review step
Gamma	Adding & Creating / Giving Feedback	View	AI content generation with creation and feedback flows	AI-first tool showing how to handle AI-generated content review
Intercom	AI workflow	View	AI content generation and approval	Major SaaS reference for AI-generated content with approval workflow

Customer.io	AI review workflow	View	AI generation with progress/review	AI-assisted content with review step — reference for progress state
Front	Empty state with CTA	View	Explanatory empty state	Reference for the "zero transcripts" empty state — clear explanation + next-step CTA
ClickUp	Empty state with CTA	View	Actionable empty state with explanation and action buttons	Reference for "zero transcripts" state with clear unblocking action
Podia	Empty state	View	Educational empty state	How a course platform handles empty content states
Teachable	Course outline editor	View	Structured content list with lessons	Direct reference for placing generated assessments on the course outline
Podia	Course outline editor	View	Course structure with sections/lessons	Competitor course outline where assessments would be inserted
Whop	Content structure editor	View	Structured content list with drag-and-drop ordering	Course/content outline ordering — reference for assessment placement UX

Built for Mars Lessons

Title	Type	BFM URL	Lesson	Relevance
A better empty state (Starling)	UX Bite	View	When Starling's AI search returns nothing, it opens onboarding listing what the search <i>can</i> and <i>can't</i> do yet — the dead end becomes a lesson in the AI's boundaries.	The zero-transcript empty state should teach what the generator works from and what it can't do yet, not just report that generation is unavailable.
Delete and redraft (Truth Social / TikTok)	UX Bite	View	Deleting a post offers "Delete & re-draft" rather than forcing a start from scratch. BFM's note: it makes deleting <i>way</i> less punishing when the content took effort to produce.	Strongest argument for a per-assessment "Delete & regenerate" alongside plain delete — the admin discards one weak question without losing the rest of the set.

Linear's "undo" warnings	UX Bite	View	Linear only interrupts an undo when it's genuinely risky — action older than 10 minutes, or triggered from a different page. Recent, in-context undos stay frictionless.	Confirm the full-set regenerate (high loss), but don't put a modal in front of deleting a single generated assessment — an undo toast is enough.
Focusing your attention on deleted items (T-Mobile)	UX Bite	View	T-Mobile inverts focus on delete — everything else goes translucent and the doomed items are highlighted, forcing attention on what's about to be discarded forever.	Apply to "Regenerate all": dim the outline and highlight the AI-generated assessments about to be replaced, so the blast radius is visible before confirming.
Contextualising your wait (Icons8)	UX Bite	View	While AI generates an image, Icons8 counts <i>up</i> rather than down — because it can't know the duration, and counting up proves the process isn't stuck.	Generation time varies with transcript length. Show lesson-by-lesson progress and elapsed time rather than a fabricated percentage or ETA.
The UX Psychology of Waiting (and Loading)	Cheatsheet	View	Variability shortens a perceived wait — give people something to read. Goodreads rotates themed quotes while loading.	The generation wait is a place to show which lesson is being read right now, rather than an undifferentiated spinner.
Turning waiting into learning (PostHog)	UX Bite	View	When a query is slow, PostHog uses the wait to teach you how to make it faster — progressive onboarding at exactly the moment it's relevant.	During generation, tell the admin that adding transcripts to the remaining lessons widens coverage next time — the one moment they care.

7. Plan of Action

Most of what this feature needs already exists in the prototype. The plan below is written against what is there — which components to reuse, and the three places where something genuinely has to change.

Phase 1: Reuse Audit & Provenance

The nine assessment types are **already built**. This phase adds no new formats.

Types	Where they already live
Fill in the Blanks, Match the Pairs, Categorise, Sequence	<code>TYPE_CONFIG</code> in <code>src/data/interactiveQuestions.ts</code> + <code>InteractiveDrawer</code> ; learner-side renderers in <code>src/pages/quiz-lab/formats/</code>
Single choice, Short text, Exercise, Poll	<code>AssessmentType</code> in <code>AddContentSidebar</code> + <code>AssessmentModal</code>
Situational tests	<code>SituationalTestDrawer</code> + the <code>SituationalTest</code> branch of <code>CreateCourse.tsx</code>

- ☐ The generator writes into the existing `ContentItem` shape (`ContentList.tsx`) — same `type` , `title` , `metadata` , `thumbnail` — so generated and manual assessments are the same object. Artwork resolves through `assessmentTypeFromLabel()` in `src/assets/assessment-illustrations/` . No parallel data model.
- ☐ Add provenance: `source?: 'manual' | 'ai'` on `ContentItem` . This is the only data change the feature needs, and FR-07, FR-09b and the outline highlight all read from it.
- ☐ **Change needed:** `Badge` has no `ai` type (`src/components/Badge/Badge.tsx` — success, warning, error, in-progress, informative, scheduled, quiz, new). Add one using the AI gradient and `SparkleIcon` , per `buttons.md` § AI Variants. This is the single net-new design-system element — get the Figma ref before building it.

Phase 2: Entry Point, Type Picker & Coverage States

- ☐ Trigger lives in `AddContentMenu` (the existing anchored picker, Figma 8695:16349) as a "Generate with AI" row — where admins already go to add content, rather than a new toolbar button. The panel's CTA is the DS AI button (`Button` variant `ai` , leading `SparkleIcon`), not a bespoke gradient.
- ☐ Type picker: Chip multi-select (`chips-switcher-tabs.md`), labelled from `TYPE_CONFIG` so the picker and the manual drawers cannot drift apart. No new picker component.
- ☐ Partial coverage (FR-04): Callout (`alerts-toast.md`) — "3 of 5 lessons have transcripts..." — not a custom strip.
- ☐ Zero transcripts (FR-05): DS Empty State (`empty-state.md` — 72px illustration, Bold-20 title, outlined + filled CTA pair, no border or fill). `src/components/mobile/EmptyState` is the mobile prototype's, not this one.

Phase 3: Generation Flow & Progress UX

- ☐ Reuse `AIWorkingCard` (`src/components/AIWorkingCard/`) — the step checklist over a gradient progress bar, already used for AI generation elsewhere. It is built to render inline where the output will land, so it drops into `ContentList` in the position the assessments will occupy and doubles as their placeholder.
- ☐ Feed it lesson names as steps, so the wait shows which lesson is being read right now (BFM loading cheatsheet).
- ☐ **Change needed:** `AIWorkingCard` takes `progress: number` (0–100) and prints a percentage. FR-11 calls for elapsed time counting up and no fabricated percentage — add an elapsed/indeterminate mode rather than inventing a number to fill the bar.
- ☐ Prerequisite: `RolePanel.tsx` still carries its own copy of this markup (`.roles-ai-working-card`). Move it onto the component while touching it — a third copy is how it stops being a component.
- ☐ Phase model (`generating` → `reviewing`) follows `AIGeneratedDrawer` . The model only: review happens on the outline (FR-06), so its one-question-at-a-time drawer is not reused here.
- ☐ Errors: existing Toast (`useToast`) `error` type, with retry returning to the same panel.

Phase 4: Review & Curation on the Outline

- ☐ Delete and "Delete & regenerate" are both rows on the card's existing `RowActionsMenu` (`danger` on delete). No new affordance on the card itself.
- ☐ **Change needed:** undo (FR-08). `ToastItem` is `{ id, type, message }` with a 5s auto-dismiss and no action slot — Toast needs an action affordance per `alerts-toast.md` before FR-08 can be built as specified.
- ☐ Regenerate all (FR-09b): existing `ConfirmModal` (already `role="alertdialog"`), with the outline behind it dimmed and the AI-generated items highlighted, so the blast radius is visible before confirming (T-Mobile). Manual items stay at full opacity — the `source` field doing visible work.
- ☐ AI badge on generated cards (Phase 1), so AI and manual assessments are distinguishable without opening anything.
- ☐ First-open onboarding explaining what the generator reads and what it cannot do yet — the same Callout, not a bespoke coach-mark (references: Starling AI empty state, Instagram contextual onboarding).
- ☐ Deliberately out: `BulkActionBar` and row multi-select. Per-item delete plus regenerate-all already covers FR-08 and FR-09; selection would add a third mental model for the same job.

Phase 5: Validation & Polish

- ☐ End-to-end across all nine types with real transcripts, confirming generated items render identically to manually created ones on the outline and in the `quiz-lab` preview.
- ☐ Edge cases: a single lesson with a transcript, very short transcripts, very long transcripts, transcripts in other languages.
- ☐ Test both regeneration loops — single-item redraft and full-set replace — for outline integrity, section placement, and correct undo behaviour.
- ☐ Dark mode and reduced motion on `AIWorkingCard`, the AI badge, and the dim-and-highlight state — semantic tokens only.
- ☐ Dogfood with course admins and collect feedback.

8. Risks & Mitigations

Risk	Impact	Mitigation
AI-generated questions contain hallucinated content not in the lesson transcripts	High — undermines admin trust and learner experience	Ground generation exclusively in transcript content (RAG-style). Apply AI-generated label so admins know to review. Research shows ~57% are directly usable; design for mandatory human review (PMC study).
Poor distractor quality in Single choice and Categorise types	Medium — makes questions too easy or nonsensical	Constrain AI prompt to derive distractors from transcript content (plausible but incorrect). Flag in onboarding that admins should review distractors carefully.
Delete-or-regenerate is too blunt — admins want to edit individual questions	Medium — frustration, workarounds (e.g., deleting all and recreating manually)	Acknowledged as a prototype constraint, softened by per-assessment "Delete & regenerate" (FR-09a) so a single bad question costs one redraft rather than the whole batch. Track edit requests to prioritise for v2.
Generation takes too long for courses with many lessons/transcripts	Medium — admin abandons the flow	Show progress indicator with per-lesson status. Consider chunking generation by lesson and streaming results. LLM/rate constraints are out of scope but should be revisited before production.
Partial transcript coverage produces an incomplete assessment set	Low — admin may not realise gaps exist	Prominently surface which lessons were included/skipped before and after generation. Never silently skip lessons.
Admins don't understand the tool or set wrong expectations	Medium — low adoption, support tickets	Use first-open onboarding that names what the generator reads and what it cannot do yet (Starling empty state). Set clear expectation: "AI drafts — review before publishing."

Situational tests have complex branching structure that AI generates poorly	Medium — the most complex assessment type may produce low-quality output	Start with simpler scenario structures (single decision point). Validate output quality for situational tests separately during Phase 5.
Generated assessments clutter the course outline	Low–Medium — admin's outline becomes unwieldy	Group or visually differentiate AI-generated assessments. Regeneration clears the previous set, preventing accumulation.